MirrorRoomSimulation

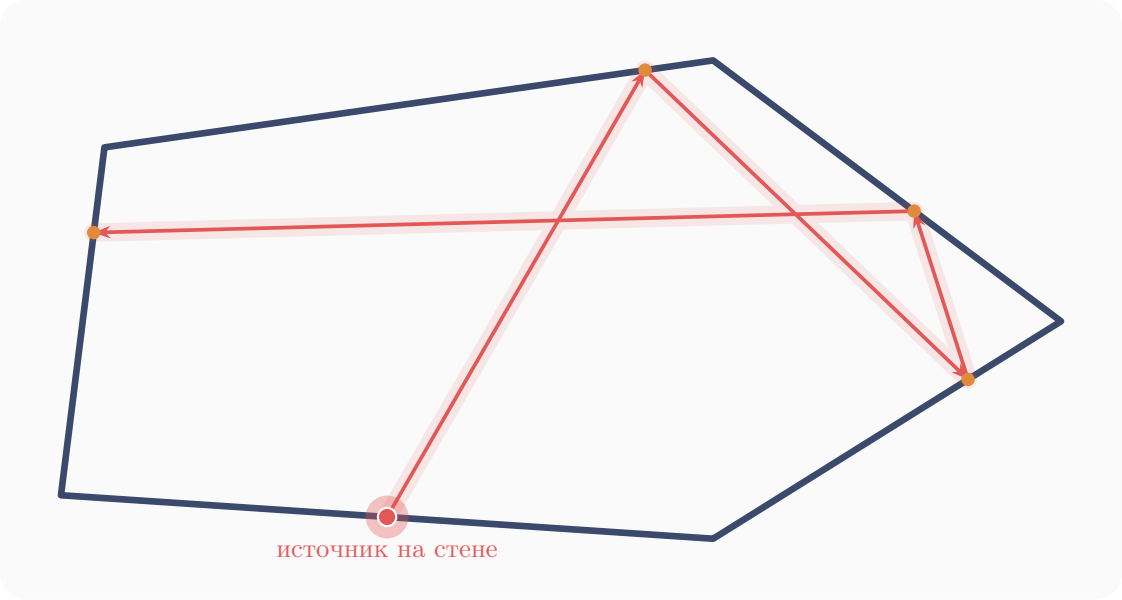
Симуляция отражений луча в зеркальной комнате

Отчёт по программному проекту

C++17 • Qt 6 Widgets

Настольное приложение для моделирования геометрической оптики

Сборка	CMake
Переводы	Qt Linguist (заготовка)
Платформы	*кроссплатформенно*



23 октября 2026 г.

Содержание

1 Общая характеристика проекта **3**

2 Возможности пользователя **3**

2.1 Основной сценарий работы . . . 3

2.2 Что может делать пользователь . . . 4

2.3 Ограничения и обратная связь . . . 5

3 Архитектура приложения **6**

3.1 Пользовательский интерфейс . . . 6

3.1.1 Кнопки по режимам . . . 7

3.2 Логика управления . . . 9

3.3 Геометрическая модель . . . 9

4 Используемые технологии и темы **10**

5 Количественные характеристики **11**

6 Состав проекта **12**

7 Руководство пользователю **12**

7.1 Построение комнаты . . . 12

7.2 Изменение зеркал . . . 13

7.3 Симуляция . . . 13

7.3.1 Начало симуляции . . . 13

7.3.2 Выбор источника . . . 13

7.3.3 Угол луча . . . 14

7.3.4 Ручная трассировка . . . 14

7.3.5 Автоматический повтор . . . 14

7.3.6 Настройки симуляции . . . 14

7.4 Сохранение истории . . . 14

7.5 Загрузка комнаты из файла . . . 15

7.5.1 Формат файла . . . 15

7.5.2 Правила разбора . . . 15

7.5.3 Тип стены . . . 16

7.5.4 Луч . . . 16

7.5.5 Пример: квадрат с лучом . . . 16
7.5.6 Пример: с дугами . . . 17
7.5.7 Порядок обхода . . . 17
7.5.8 Типичные ошибки . . . 17
7.6 Справочник констант и формул . . . 17

8 Руководство программисту **18**
8.1 Сборка проекта . . . 18
8.2 Используемые библиотеки . . . 19
8.3 Где менять параметры . . . 21
8.4 Основные функции проекта . . . 21
8.5 На что обращать внимание . . . 23
8.6 Геометрия и константы — подробнее . . . 23
8.7 Алгоритм загрузки комнаты из файла . . . 24

9 Заключение **25**

1 Общая характеристика проекта

MirrorRoomSimulation — интерактивный симулятор геометрической оптики, который показывает, как световой луч многократно отражается от стен замкнутой комнаты произвольной формы. Пользователь строит комнату щелчками мыши или загружает из txt-файла, назначает стенам тип зеркала (плоское, вогнутое, выпуклое) и наблюдает за поведением луча: программа рассчитывает каждое отражение по закону отражения, анимирует отрезки пути и позволяет сохранить историю в текстовый файл.

Идея приложения

- пользователь строит комнату из точек и стен (или загружает её из файла);
- стены могут быть плоскими, вогнутыми или выпуклыми зеркалами;
- после замыкания комнаты пользователь размещает источник луча на одной из стен;
- луч отражается от стен по закону отражения;
- программа показывает траекторию, анимирует её и сохраняет историю отражений.

Проект можно рассматривать одновременно как:

- пользовательский интерактивный симулятор;
- учебный пример геометрических расчетов;
- пример организации кода на C++ и Qt;
- основу для дальнейшего развития оптического моделирования.

2 Возможности пользователя

2.1 Основной сценарий работы

Работа пользователя состоит из нескольких этапов: построение комнаты (мышью или загрузкой из файла), замыкание контура, установка источника луча, задание угла, трассировка отражений и сохранение результата. Из режима трассировки можно вернуться к редактированию или замыканию комнаты (штриховые стрелки на рисунке 1): при этом путь луча сбрасывается, но уже выбранный источник сохраняется. Если комната загружена из файла вместе с источником, этапы «замыкание» и «установка источника» пропускаются — сразу включается режим трассировки.

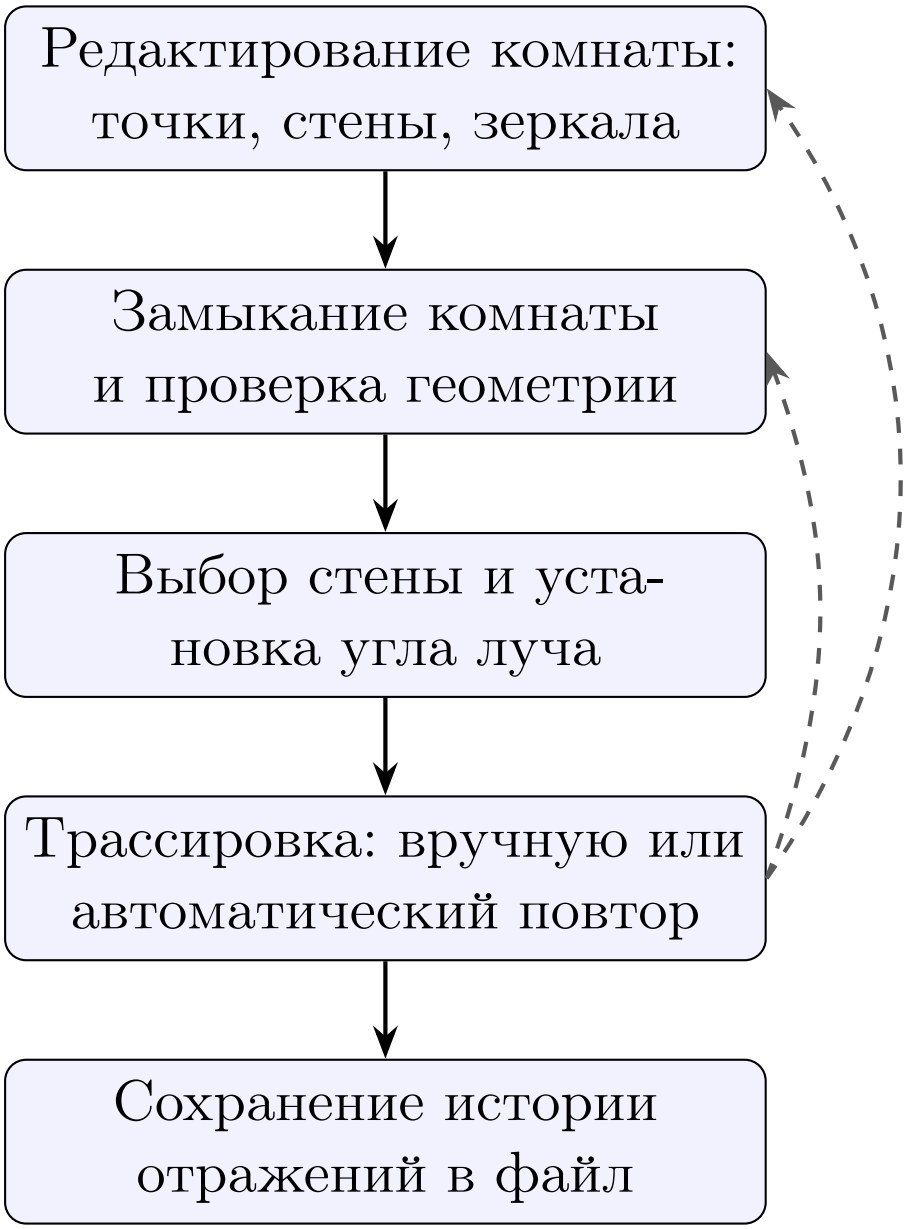


Рис. 1: Основной пользовательский сценарий. Штриховые стрелки — возврат к редактированию или замыканию комнаты из режима трассировки. Альтернатива этапам «замыкание» и «выбор источника»: кнопка 3 в режиме редактирования «Загрузить значения из файла» — файл может содержать и комнату, и источник, тогда сразу включается трассировка.

2.2 Что может делать пользователь

В режиме редактирования комнаты:

- ставить новые точки кликом по рабочей области;
- изменять существующие точки (координаты, тип зеркала, радиус кривизны);
- удалять точки;
- откатывать последнюю поставленную точку;

- замыкать комнату (кнопкой «Начать симуляцию» или кнопкой «Подключить» в диалоге точки);
- **загружать комнату (и, если задан, источник луча) из txt-файла**;
- **просматривать справку о формате входного файла**.

В режиме симуляции:

- размещать источник луча на стене;
- задавать угол луча относительно стены;
- выполнять отражения вручную (по одному);
- запускать и останавливать автоматический повтор отражений;
- менять скорость анимации луча;
- менять задержку между отражениями в авторежиме;
- включать режим удаления старых отражений с ограничением их количества на экране;
- сохранять историю отражений в текстовый файл.

2.3 Ограничения и обратная связь

Программа ограничивает действия пользователя, чтобы геометрия оставалась корректной.

Ключевые ограничения

- рабочая область имеет размеры от 0 до 1580 по X и от 0 до 1000 по Y;
- количество сторон комнаты должно быть от 4 до 9;
- стены не должны пересекаться;
- стены не должны касаться друг друга вне общих вершин;
- точки не должны находиться слишком близко друг к другу;
- источник луча должен располагаться на стене;
- источник луча не может находиться слишком близко к вершине;
- угол луча должен быть от 0,01° до 179,99°;
- радиус криволинейного зеркала должен быть не меньше половины длины соответствующей стены;
- файл для загрузки комнаты — UTF-8, не больше 1 МиБ.

Все ошибки и предупреждения выводятся в служебный чат справа. Это позволяет пользователю понимать, почему действие не было выполнено.

3 Архитектура приложения

Проект удобно представить как дерево зависимостей: каждая стрелка идёт от того, кто вызывает, к тому, кого вызывают. На вершине — интерфейс (`MainWindow` и диалоги), в центре — оркестратор `FrontEndOperators`, внизу — геометрическая модель (`Polygon`, `Mirror`, `Points`). Сбоку показаны «сквозные» сущности: константы и стиль (штриховые стрелки) и типы данных (пунктирные стрелки).

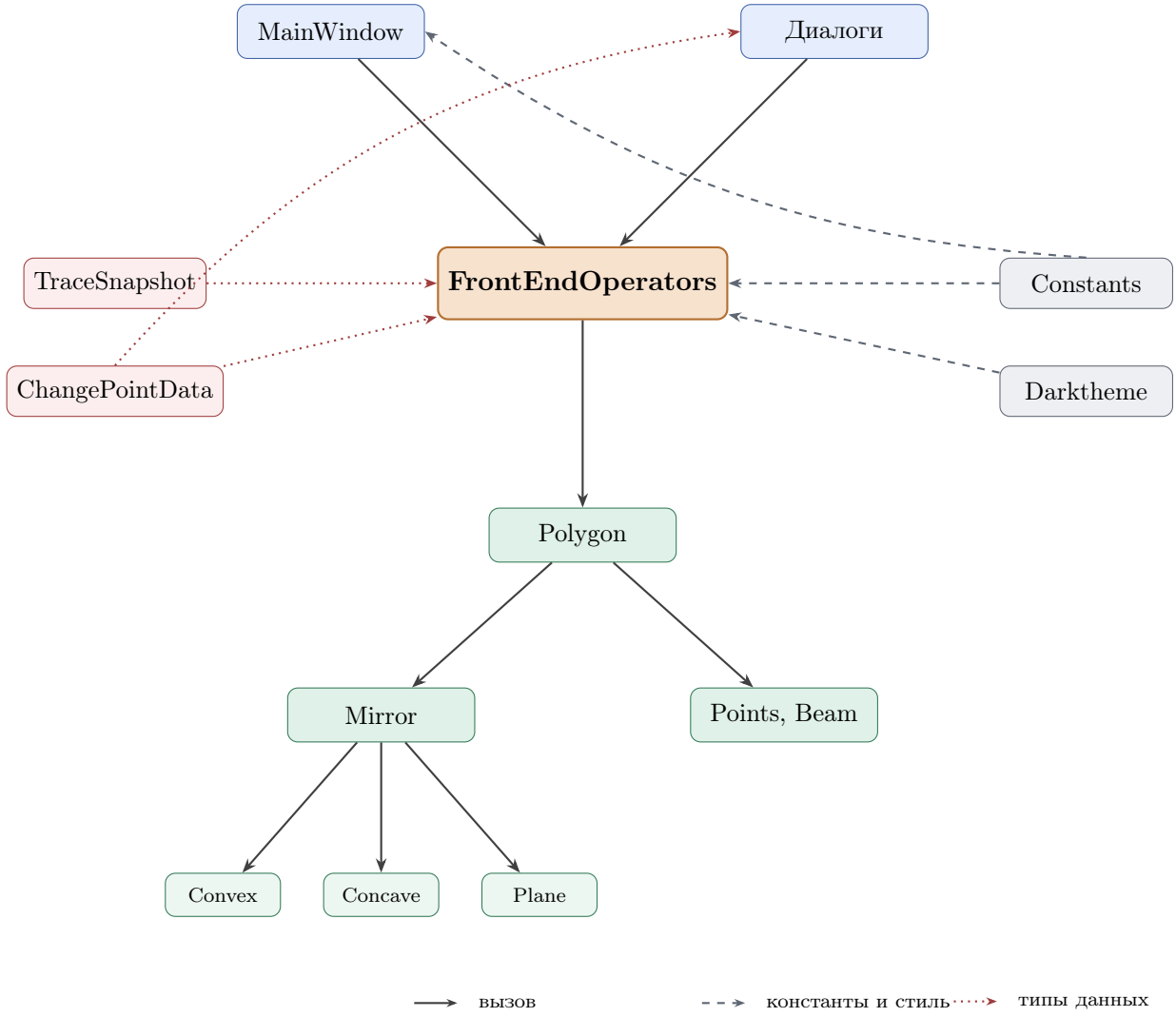


Рис. 2: Дерево зависимостей проекта. `FrontEndOperators` — оркестратор: он связывает интерфейс и модель.

3.1 Пользовательский интерфейс

Интерфейс включает: главное окно с рабочей областью; служебный чат справа; кнопки управления режимами; ползунки скорости и задержки; галочку «Удалять старые отражения» и поле «Макс. кол-во»; модальные диалоги для ввода параметров.

Главное окно самостоятельно рисует сцену через `QPainter`: фон, рабочую зону, стены, точки, траекторию луча и источник.

3.1.1 Кнопки по режимам

Набор доступных кнопок зависит от текущего режима; недоступные кнопки приглушаются.

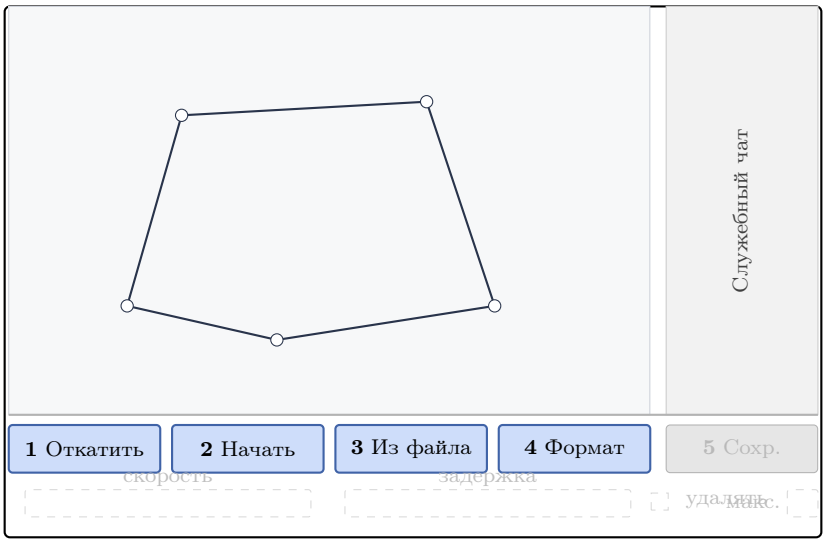
В режиме редактирования комнаты:

- **«Откатить точку»** (кнопка 1) — удаляет последнюю вершину цепочки. Если комната была замкнута, действие убирает замыкающую стену и возвращает в редактирование.
- **«Начать симуляцию»** (кнопка 2) — замыкает комнату (если ещё не замкнута) и переключает в режим симуляции.
- **«Загрузить значения из файла»** (кнопка 3) — открывает диалог выбора txt-файла. Комната и (если в файле задан) источник луча заменяют текущую сцену; при ошибке ничего не меняется. Всегда активна.
- **«Формат входного файла»** (кнопка 4) — открывает справочный диалог о формате txt-файла для загрузки. Всегда активна.
- **«Сохранить историю. . . »** (кнопка 5) — приглушена, пока нет сохранённой истории; активна, если история есть (например, после возврата в редактирование).
- Ползунки скорости и задержки, галочка и поле «Макс. кол-во» — скрыты (это настройки симуляции).

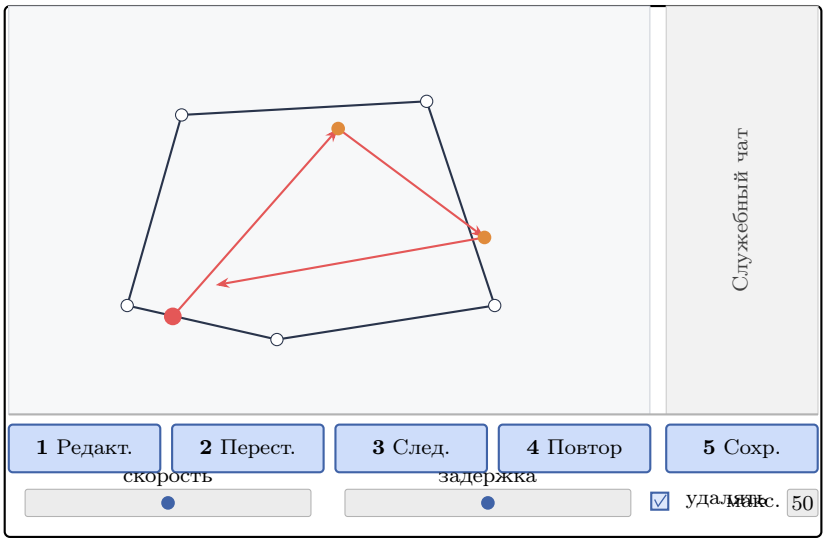
В режиме симуляции (после выбора источника):

- **«Редактировать комнату»** (кнопка 1) — переход в режим редактирования. Путь луча сбрасывается, источник сохраняется.
- **«Переставить источник света»** (кнопка 2) — удаляет текущий источник и включает режим выбора нового.
- **«Следующее отражение»** (кнопка 3) — рассчитывает и анимирует один отрезок пути.
- **«Повтор анимации»** (кнопка 4) — запускает автоматический повтор; во время повтора текст меняется на «Остановить повтор».
- **«Сохранить историю. . . »** (кнопка 5) — открывает диалог выбора точности и сохранения файла.
- Ползунки, галочка и поле «Макс. кол-во» — активны, если не идёт анимация/повтор.

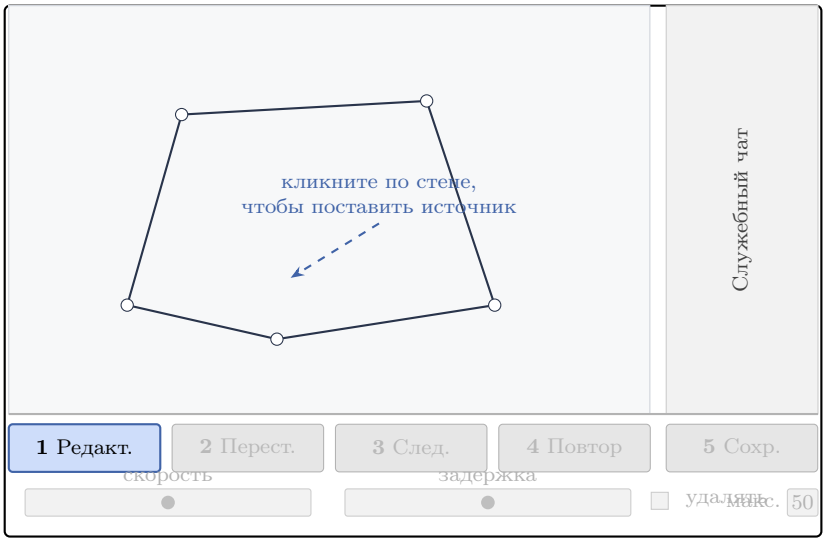
В режиме выбора источника (клик по стене): активна только «Редактировать комнату» (кнопка 1); остальные кнопки и настройки заблокированы — сначала выберите стену и задайте угол (или отмените диалог).



(а) Режим редактирования комнаты



(б) Режим симуляции



(в) Режим выбора источника

Рис. 3: Схематическое изображение интерфейса в трёх режимах. **Синие** кнопки — активны, **серые** — недоступны (приглушены), **штриховые** — скрыты. Номера соответствуют п. 3.1.1. В режиме редактирования кнопки 3 и 4 теперь активны: загрузка комнаты из файла и справка о формате.

3.2 Логика управления

Класс `FrontEndOperators` отвечает за: создание, удаление и изменение точек; замыкание комнаты с проверкой геометрии; расчёт следующего отражения; обнаружение зацикливания луча; хранение истории отражений и снимка комнаты; загрузку комнаты и луча из txt-файла (`LoadRoom`). Связь с интерфейсом — через `std::function notify`.

3.3 Геометрическая модель

Геометрическая модель включает: класс точки `Points`; класс луча `Beam`; абстрактный класс зеркала `Mirror`; реализации плоского, вогнутого и выпуклого зеркал; класс многоугольника `Polygon`. В этом слое выполняются расчёты пересечений, нормалей, отражений и проверки корректности комнаты.

Ключевые формулы геометрической модели

Расстояние: $d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Отражение: $\mathbf{d}' = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\,\mathbf{n}$

Нормаль к плоскому зеркалу: $\mathbf{n} = \dfrac{(-d_y,\ d_x)}{\sqrt{d_x^2 + d_y^2}}, \quad (d_x, d_y) = (x_e - x_s,\ y_e - y_s)$

Нормаль к дуге: $\mathbf{n} = \dfrac{P_{\text{hit}} - C}{\|P_{\text{hit}} - C\|}$

Центр дуги: $h = \sqrt{r^2 - (d/2)^2}, \quad C = M \pm h \cdot \mathbf{n}_\perp$

Ближайшая точка отрезка: $t^* = \dfrac{(P - A) \cdot (B - A)}{\|B - A\|^2}, \quad Q = A + \text{clamp}(t^*, 0, 1)\,(B - A)$

Луч и дуга: $\|P + t\,\mathbf{d} - C\|^2 = r^2$

Ограничение дуги: $|r| \geq d/2$

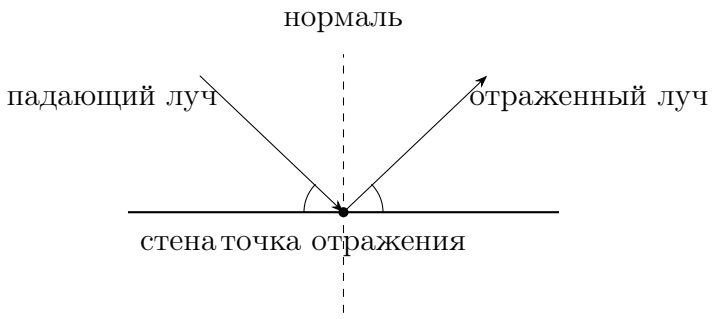


Рис. 4: Схема отражения луча от стены (углы падения и отражения равны)

Иерархия наследования

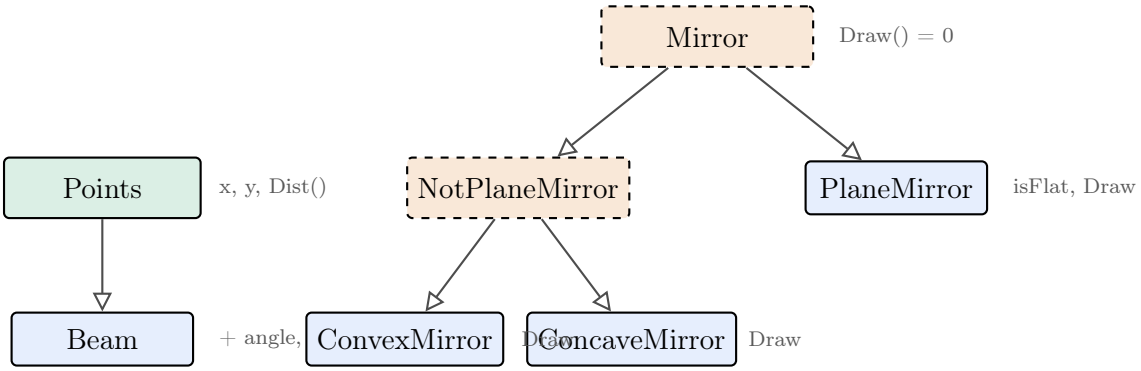


Рис. 5: Иерархия наследования. Пунктирная рамка — абстрактный класс. Треугольник — `public`-наследование.

4 Используемые технологии и темы

- C++17;
- Qt 6 Widgets;
- объектно-ориентированное программирование;
- наследование и полиморфизм для зеркал;
- умные указатели для управления стенами;
- сигнально-слотовая модель Qt;
- пользовательская отрисовка через `QPainter`;
- анимация через `QVariantAnimation`;
- таймеры через `QTimer`;
- диалоговые окна и модальный ввод;
- файловый ввод-вывод через `QFile` и `QTextStream`;
- сборка через CMake;
- заготовка локализации через Qt Linguist.

Отдельно стоит отметить вычислительную геометрию: пересечение отрезков, пересечение отрезков и дуг, пересечение луча с отрезком и дугой, вычисление нормалей, отражение направления, проверку многоугольника на самопересечения.

5 Количественные характеристики

В этом разделе приведены три диаграммы, дающие представление о масштабе проекта: распределение строк кода по темам, по модулям и связь скорости луча со временем анимации.

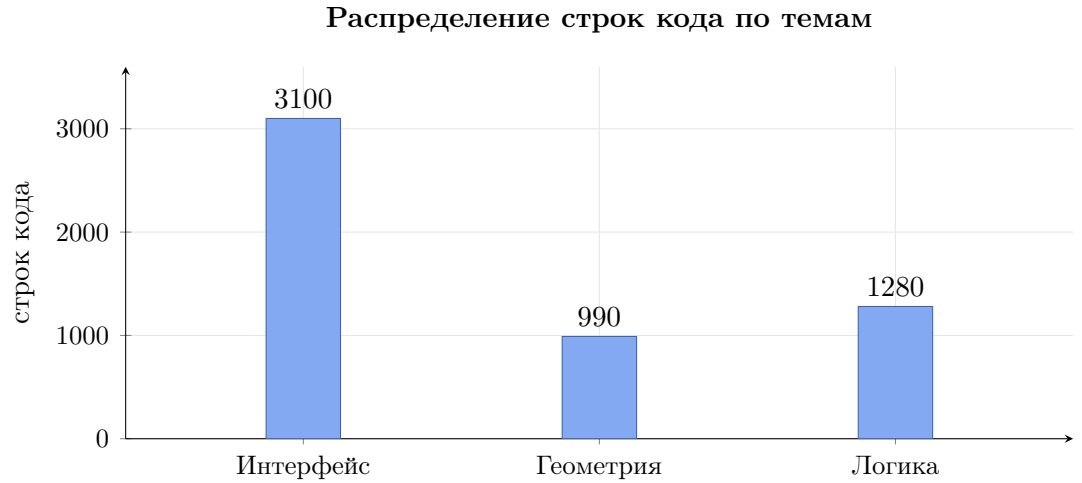


Рис. 6: Распределение строк кода по трём основным слоям проекта. Вспомогательные файлы (`constants.h`, `CMakeLists.txt`, файл перевода, `README`) в диаграмму не включены.



Рис. 7: Объём кода по модулям (`.cpp` + `.h`). `mainwindow` включает `.ui`-файл; `frontendoperators` включает `LoadRoom`; новый модуль `formathelpdialog`.

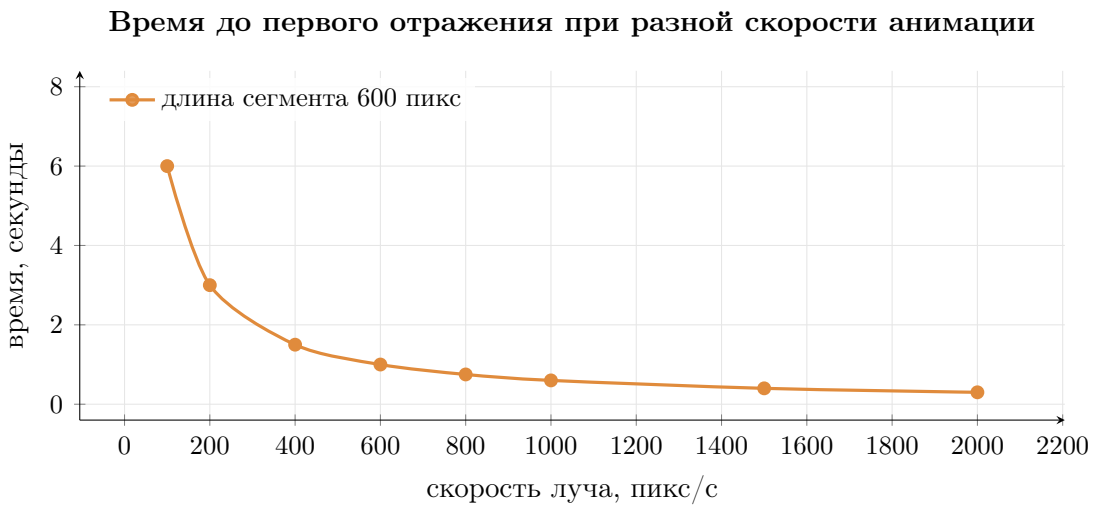


Рис. 8: Время анимации одного сегмента обратно пропорционально скорости луча. Пользователь регулирует скорость ползунком в пределах 100–2000 пикс/с с шагом 50.

6 Состав проекта

Группа	**Файлы и роль**
Точка входа	`main.cpp`: создание приложения, попытка загрузки перевода, запуск главного окна.
Интерфейс	`mainwindow.ui`, `mainwindow.h`, `mainwindow.cpp`; диалоги: `beamangledialog.*`, `changepointdialog.*`, `changeorconnectdialog.*`, `savehistorydialog.*`, `formathelpdialog.*`.
Логика управления	`frontendoperators.h`, `frontendoperators.cpp`: операции создания, удаления, изменения точек, замыкания, трассировки и загрузки комнаты из файла.
Геометрическая модель	`polygon.h`, `polygon.cpp`, `mirror.h`, `mirror.cpp`, `points.h`, `points.cpp`.
Константы и тема	`constants.h`: пределы, цвета, допуски, параметры анимации. `Darktheme.h`: стиль главного окна и выпадающих списков.
Сборка и переводы	`CMakeLists.txt`; файл перевода `MirrorRoomSimulation_ru_RU.ts`; `README.md`.

7 Руководство пользователю

7.1 Построение комнаты

После запуска программа находится в режиме редактирования. Пользователь кликает по рабочей области, чтобы поставить точки. Между точками автоматически создаются

плоские стены. Первый клик задаёт первую вершину; каждая следующая точка добавляет стену от предыдущей вершины к новой.

Ограничения по числу сторон (4–9) проверяются в момент замыкания. Если набрано меньше 4 или больше 9 вершин, кнопка «Начать симуляцию» выдаст ошибку в чат, и комната останется незамкнутой.

Клик по существующей точке открывает диалог «Изменить / Подключить / Удалить»:

- **«Изменить»** — открывает диалог с координатами, типом зеркала и радиусом кривизны;
- **«Подключить»** — замыкает комнату на выбранной вершине (удаляет все стены после неё и добавляет замыкающую);
- **«Удалить»** — удаляет точку. Для крайних точек удаляется и соответствующая стена; для средней стены слева и справа заменяются одной плоской.

В уже замкнутой комнате пункт «Подключить» недоступен.

7.2 Изменение зеркал

В диалоге изменения точки можно задать: координату X (0–1580), координату Y (0–1000), тип зеркала (плоское, вогнутое или выпуклое), радиус кривизны (положительное число).

Про радиус кривизны

Если введённый радиус меньше половины длины стены, кривое зеркало невозможно. Программа оставит плоское зеркало и выведет предупреждение в чат. Если новое положение точки приводит к пересечению или слишком близкому соседству стен, изменение отменяется с сообщением в чат.

7.3 Симуляция

7.3.1 Начало симуляции

При нажатии «Начать симуляцию» программа: (1) проверяет количество сторон (4–9); (2) замыкает комнату (если ещё не замкнута); (3) проверяет, что стены не пересекаются и не касаются друг друга вне общих вершин; (4) переключает интерфейс в режим симуляции. Если источник уже стоял на стене, он сохраняется вместе с углом. Иначе включается режим выбора источника.

7.3.2 Выбор источника

В режиме выбора источник ставится кликом по стене. Клик должен попасть в пределы 6 пикселей от линии зеркала. Программа находит ближайшую точку на стене и проверяет, что она не ближе 10 пикселей к вершинам. Затем открывается диалог `BeamAngleDialog`.

7.3.3 Угол луча

Угол измеряется между стеной и лучом в сторону комнаты: 90° — перпендикулярно стене; допустимый диапазон — 0,01° … 179,99° (2 знака после запятой); для криволинейной стены берётся касательная в точке источника. Ввод проверяется по набранному тексту. Изменить угол можно в любой момент: клик по красной точке источника (в пределах 5 пикселей) снова открывает диалог.

7.3.4 Ручная трассировка

Нажатие «Следующее отражение» выполняет один шаг физики: (1) из текущей точки по направлению луча ищется ближайшее пересечение со стенами; (2) в найденной точке вычисляется нормаль; (3) по закону отражения $\mathbf{d}' = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$ находится новое направление; (4) отрезок анимируется с учётом текущей скорости. Если луч уже зациклился, кнопка становится недоступной, а в чат уходит предупреждение с периодом цикла.

7.3.5 Автоматический повтор

Кнопка «Повтор анимации» запускает серию отражений с паузой, заданной вторым ползунком. Повторное нажатие просит остановку после текущей анимации; если идёт пауза, повтор останавливается сразу. Повтор также останавливается автоматически при зацикливании луча.

7.3.6 Настройки симуляции

- **Скорость луча** — 100 … 2000 пикс/с, шаг 50. Влияет на длительность анимации: время = длина / скорость.
- **Задержка** — 0 … 5000 мс, шаг 100. Пауза между отражениями в авторежиме.
- **Галочка «Удалять старые отражения»** — на экране остаются только последние N отрезков.
- **Поле «Макс. кол-во отражений»** — задаёт N (от 1 до 1 000 000); применяется по Enter или при потере фокуса.

Все настройки блокируются во время анимации и повтора.

7.4 Сохранение истории

Кнопка «Сохранить историю… » активна, когда: (1) в истории есть хотя бы одно отражение; (2) не идёт анимация; (3) не идёт повтор; (4) не выбран режим выбора источника.

Когда кнопка недоступна и когда файл не сохранится

Кнопка приглушена (нажатие с сообщением в чат), если: не сделано ни одного отражения; идёт анимация или повтор; выбран режим выбора источника.
Даже когда кнопка активна, файл может не сохраниться, если: отменён диалог выбора точности; отменён стандартный диалог сохранения; указано имя без `.txt`, а такой `.txt` уже существует; не хватило прав на запись или диск переполнен.
История хранится и после `ResetTrace`: после сброса визуального пути кнопка продолжает работать и сохраняет «прошлую» симуляцию.

При нажатии открывается `SaveHistoryDialog` с выбором точности (0–17 знаков). Затем — стандартный диалог сохранения. В файл попадают: дата и время; структура комнаты (вершины + типы зеркал); параметры источника луча; точки отражений; углы направления луча после каждого отражения (в градусах, 0–360).

7.5 Загрузка комнаты из файла

Комнату можно не строить мышью, а описать в обычном текстовом файле и загрузить. Кнопка 3 в режиме редактирования (**«Загрузить значения из файла»**) открывает диалог выбора txt-файла; кнопка 4 (**«Формат входного файла»**) показывает эту же справку в виде диалога.

Главное

Загруженная комната проходит все те же проверки, что и построенная мышью. Если файл содержит ошибку — текущая комната не меняется, а причины выводятся в служебный чат. При успехе комната заменяется целиком и сразу включается симуляция: если в файле есть луч и он лежит на стене, доступны «Следующее отражение» и «Повтор анимации»; иначе включается режим выбора источника — нужно кликнуть по стене.

7.5.1 Формат файла

UTF-8 с BOM или без, размер не больше 1 МиБ (строго больше — отказ). Секция углов обязательна, секция луча — нет; порядок секций произвольный. Строки, не похожие на данные, игнорируются и могут служить комментариями.

Секция «Углы комнаты:». Строка с фразой «Углы комнаты:», а сразу под ней — от 4 до 9 строк подряд, по два числа в каждой (x и y вершины). После координат можно (необязательно) указать тип стены.

Секция «Луч:». Строка с фразой «Луч:», а сразу под ней одна строка с двумя числами (координаты источника) или тремя (x, y, угол).

7.5.2 Правила разбора

- Число — это одна или несколько цифр, при желании с дробной частью через точку или запятую (`400`, `400.5`, `400,5`).
- Знак числа не учитывается: `-100` читается как `100`.

- Всё остальное (буквы, скобки, `;`, `=`) — разделители и мусор. `(400; 100)` и `x=400 y=100` читаются как `400` и `100`.
- Первая не значащая строка обрывает список углов (даже пустая).
- Степенной записи нет: `1e2` — это два числа `1` и `2`.
- `400,100` без пробела — одно число `400.1`; пару с запятой пишите через пробел: `400, 100`.

7.5.3 Тип стены

- `тип: вогнутое(R)` или `тип: выпуклое(R)`; допускаются окончания «-ая», синоним «выгнутое».
- Радиус R обязателен и только в круглых скобках; всё после закрывающей скобки игнорируется.
- Тип относится к стене от этой вершины к следующей; без записи типа стена плоская.
- R не меньше половины длины стены и не больше 1 000 000.

7.5.4 Луч

- Точка ищется как при клике мышью: ближайшая стена в пределах 6 пикселей, не ближе 10 пикселей к вершине.
- Угол — между стеной и лучом, $0{,}01° \ldots 179{,}99°$; если не указан — $90°$.
- Негодный луч файл не отклоняет: комната загрузится без луча, нужно будет кликнуть по стене.

7.5.5 Пример: квадрат с лучом

```
Углы комнаты:
400 100
700 100
700 400
400 400
Луч:
550 100 45
```

Источник на верхней стене, луч уходит в комнату под $45°$ к стене по ходу обхода (вправо и вниз).

7.5.6 Пример: с дугами

```
Углы комнаты:
400 100
600 100
700 300 тип: вогнутое(300)
600 500
400 500 тип: выпуклое(250)
Луч:
500 100 60
```

5 вершин и 5 стен. Стена 3 идёт от (700,300) к (600,500) и вогнутая, $R = 300$. Стена 5 идёт от (400,500) к (400,100) и выпуклая, $R = 250$. Остальные плоские. Источник на плоской стене 1 в точке (500,100), угол 60°.

7.5.7 Порядок обхода

Вершины стоит перечислять по часовой стрелке. Код считает «внутри» слева от направления обхода: у выпуклой стены центр окружности слева, у вогнутой справа. Если перечислить против часовой, типы по виду меняются местами: «выпуклое» будет вдавлено внутрь, «вогнутое» — выпячено наружу. Лучи при этом считаются по нарисованной геометрии, то есть согласованно с картинкой.

7.5.8 Типичные ошибки

Частые проблемы

- **Пустая строка внутри углов** обрывает список — остальные точки не прочитаются.
- **Строка «Точка 1: (400; 100)»** содержит три числа и не подходит как угол. Именно поэтому файл истории, сохранённый кнопкой 5, загрузить обратно нельзя — у него другой формат.
- **400,100 без пробела** — одно число `400.1`, строка не значащая.
- **Кодировка не UTF-8** даёт ошибку «нет ни фразы...».
- **Строка после «Луч:» должна идти сразу**: пустая строка между ними — ошибка.

7.6 Справочник констант и формул

Параметры интерфейса

Параметр	Диапазон	Шаг / по умолчанию
Угол луча к стене	0,01° … 179,99°	2 знака
Скорость луча	100 … 2000 пикс/с	50
Задержка между отражениями	0 … 5000 мс	100
Стен в комнате	4 … 9	—
Рабочая область	1580 × 1000 пикселей	—
Точек в истории	1 … 1 000 000 (на экране)	50
Точность сохранения	0 … 17 знаков	3
Радиус клика по точке	5 пикселей	—
Допуск при постановке источника	6 пикселей	—
Минимальное расстояние между точками	10 пикселей	—
Размер файла загрузки	не больше 1 МиБ	—
Радиус кривой стены	$\frac{1}{2}$ хорды … 1 000 000	—
Угол по умолчанию (файл)	90°	—

Формулы

Отражение: $\mathbf{d}' = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\,\mathbf{n}$

Нормаль к плоской стене: $\mathbf{n} = \dfrac{(-d_y,\ d_x)}{\|(d_x, d_y)\|}$

Нормаль к дуге: $\mathbf{n} = \dfrac{P_{\text{hit}} - C}{\|P_{\text{hit}} - C\|}$

Центр дуги: $C = M \pm \sqrt{r^2 - (d/2)^2} \cdot \mathbf{n}_\perp$

Угол луча к стене: $\theta = \angle(\mathbf{t},\, \mathbf{d}), \quad \mathbf{d} = \cos\theta\,\mathbf{t} + \sin\theta\,\mathbf{n}_{\text{комн.}}$

8 Руководство программисту

8.1 Сборка проекта

Для сборки необходимы: компилятор с поддержкой C++17, CMake и Qt 6 с модулями Widgets и LinguistTools.

```
cmake -S . -B build
cmake --build build
```

После сборки исполняемый файл будет находиться в каталоге сборки.

8.2 Используемые библиотеки

В проекте применяются три группы библиотек: модули Qt, стандартная библиотека C++ и собственные заголовки проекта.

Модули Qt 6

Модуль	Где используется
`Qt6::Widgets`	`mainwindow.*`, все диалоги; всё, что связано с виджетами, отрисовкой, вводом.
`Qt6::Core`	Неявно через `Qt6::Widgets`: `QFile`, `QTextStream`, `QDateTime`, `QStandardPaths`, `QVariantAnimation`, `QTimer`.
`Qt6::Gui`	Неявно: `QPainter`, `QPen`, `QBrush`, `QPolygonF`, `QPainterPath`, `QStyledItemDelegate`.
`Qt6::LinguistTools`	`CMakeLists.txt`: `qt_add_translations`.

Классы Qt, подключаемые через заголовки

Заголовок	Роль
`QApplication`, `QMainWindow`	точка входа, главное окно.
`QDialog`, `QWidget`	все диалоги и виджеты.
`QLabel`, `QPushButton`, `QLineEdit`	элементы интерфейса.
`QDoubleSpinBox`, `QSpinBox`, `QComboBox`	поля ввода чисел и списки.
`QStyledItemDelegate`	тёмный выпадающий список в `ChangePointDialog`.
`QTextEdit`, `QCheckBox`, `QSlider`, `QScrollArea`	настройки симуляции, чат.
`QVBoxLayout`, `QHBoxLayout`, `QLayoutItem`	компоновка.
`QPainter`, `QPen`, `QPolygonF`, `QPainterPath`	отрисовка сцены.
`QEvent`, `QKeyEvent`, `QMouseEvent`	фильтр событий и клики.
`QTimer`, `QVariantAnimation`, `QGraphicsOpacityEffect`	анимация, паузы, приглушение.
`QFile`, `QFileDialog`, `QFileInfo`, `QTextStream`	сохранение и загрузка.

Заголовок	Роль
QStringConverter	кодировка UTF-8 при записи истории (QTextStream::setEncoding).
QStandardPaths, QDateTime, QTime	папка по умолчанию, метки времени.
QLocale, QTranslator	загрузка перевода при старте.
QMessageBox, QScrollBar	редкие вспомогательные виджеты.

Стандартная библиотека C++

Заголовок	Где используется
<cmath>	std::sqrt, std::atan2, std::cos, std::sin, std::fmod, std::abs.
<algorithm>	std::clamp, std::min, std::max, std::find.
<memory>	std::unique_ptr, std::make_unique.
<vector>	все коллекции точек, стен, истории.
<stdexcept>	std::invalid_argument в конструкторе Mirror.
<functional>	std::function для notify.
<utility>	std::pair, std::move.
<limits>	std::numeric_limits, max_digits10.
<type_traits>	метапрограммирование в заголовках.
<string>	std::string, std::u32string в загрузчике.
<sstream>, <iomanip>	std::istringstream при разборе чисел, форматирование координат.
<locale>	std::locale::classic() — разбор чисел не зависит от системной локали.
<fstream>, <iostream>	задел на файловый ввод-вывод.

Собственные заголовки

Заголовок	Роль
points.h	Points, Beam.
mirror.h	Mirror и наследники.
polygon.h	Polygon.
frontendoperators.h	контроллер + TraceSnapshot + LoadResult.
constants.h	все числовые константы и цвета.
Darktheme.h	QSS-строка для главного окна, стиль выпадающих списков.

Заголовок	Роль
beamangledialog.h, changepointdialog.h, changeorconnectdialog.h, savehistorydialog.h, formathelpdialog.h	диалоги.
mainwindow.h, ui_mainwindow.h	главное окно (второй генерируется из .ui).

8.3 Где менять параметры

Большинство настраиваемых значений находятся в constants.h: размеры рабочей области, минимальные расстояния, диапазон угла луча, скорость анимации, параметры чата, допуски, цвета темы, лимиты загрузки файла.

8.4 Основные функции проекта

MainWindow (mainwindow.cpp)

Функция	Роль
mousePressEvent	Диспетчер кликов: постановка новой точки, клик по вершине, по стене, по красной точке источника.
paintEvent	Отрисовка сцены: фон, стены, путь луча, точка источника.
UpdateButtons	Управление доступностью и приглушением кнопок и ползунков; меняет текст кнопок 3 и 4 в зависимости от режима.
AddMessage	Добавление «пузыря» в чат с уровнем и временем.
FirstButClicked–FifthButClicked	Обработчики кнопок 1–5.
LoadRoomFromFile	Кнопка 3 в режиме редактирования: диалог выбора txt-файла, чтение, вызов FrontEndOperators::LoadRoom, при успехе — SecondButClicked.
StartReflectionStep	Один шаг NextReflection + запуск анимации отрезка.
FinishReflection	Конец анимации: показ отложенных сообщений и следующий шаг.
eventFilter	Enter/потеря фокуса в поле, блокировка с сообщением.
CommitTraceLimit	Применение числа из поля «Макс. кол-во».
IgnoreDuringAnimation	Проверка «идёт ли анимация/повтор».

FrontEndOperators (frontendoperators.cpp)

Функция	Роль
`CreatePoint`	Добавляет новую вершину в конец цепочки.
`ConnectPoints`	Замыкает комнату на указанной вершине.
`Binding`	Вызов `ConnectPoints(0)` — стандартное замыкание.
`DeleteLast`	Удаляет последнюю стену (или первую вершину).
`DeletePoint`	Удаляет конкретную вершину; для средней — заменяет две стены одной плоской.
`ChangeMirror`	Изменяет координаты, тип и радиус стены.
`LoadRoom`	Разбор и проверка txt-файла: UTF-8 → UTF-32, две секции, валидация значений, построение `Polygon`, проверка `CheckWalls` и `Check`, опционально `PlaceBeam` и `SetBeamAngle`. Возвращает `LoadResult`.
`NextReflection`	Один шаг физики: поиск пересечения, отражение, проверка зацикливания.
`ResetTrace`	Сброс видимого пути.
`HasTrace`	`true`, если есть что показывать.
`MakeSnapshot`	Снимок комнаты и источника для сохранения истории.
`Notify`	Отправка сообщения в интерфейс.

Polygon (polygon.cpp)

Функция	Роль
`isExist`, `isCycled`, `Size`	Состояние контура.
`PlaceBeam`	Постановка источника на ближайшую стену (в пределах допуска).
`SetBeamAngle`	Вычисление направления луча по углу к стене.
`Check`	Проверка замкнутости, непрерывности, отсутствия самопересечений.
`TraceBeamIter`	Один шаг физики: поиск ближайшего зеркала, нормали, отражённого направления.

Mirror и наследники (mirror.cpp / mirror.h)

Функция	Роль
`getStart`, `getEnd`	Концы хорды.
`getRad`, `getRadius`	Радиус со знаком и его модуль.
`Dist`	Длина хорды.
`isFlat`	Плоское ли зеркало.

Функция	Роль
`CircleCenter()`	Центр окружности по знаку радиуса.
`CircleCenter(bool)`	Центр с явно заданной стороной.
`flatPointOnArc`, `arcPointOnArc`	Точка на отрезке/дуге (`protected`).
`PointOnArc`	Публичная проверка «точка лежит на зеркале».
`ClosestPoint`	Ближайшая точка зеркала к заданной.
`Intersections`	Пересечения с другим зеркалом.
`Draw`	Отрисовка через `QPainter`.

FormatHelpDialog (formathelpdialog.cpp)

Функция	Роль
`FormatHelpDialog`	Конструктор: собирает HTML-текст справки по текущим значениям констант (`Limits`, `BeamAngle`), показывает пример файла.
`accept`	Закрытие по кнопке «Закрыть» или Esc.

8.5 На что обращать внимание

- геометрические расчёты зависят от числовых допусков (`GeoEps`, `FeEps`);
- изменение стен требует проверки пересечений (`CheckWalls`);
- изменение луча должно сбрасывать текущую трассировку (`ResetTrace`);
- во время анимации многие действия должны блокироваться;
- история отражений хранится даже после сброса визуального пути;
- пользовательские сообщения должны объяснять причину отказа;
- загрузка комнаты из файла атомарна: комната собирается во временном объекте и подменяет текущую только после всех проверок.

8.6 Геометрия и константы — подробнее

Допуски геометрии (`GeoEps`)

- `EPS` $= 10^{-9}$ — общий допуск в `mirror.cpp` и `polygon.cpp`.
- `POINT_EPS` $= 10^{-6}$ — допуск на совпадение точек.
- `PAR_EPS` $= 10^{-10}$ — допуск параллельности в `IntersectSegments`.
- `FLAT_R` $= 10^{7}$ — порог «это уже прямая» в `Mirror::isFlat`.
- `TRACE_EPS` $= 10^{-7}$ — минимальный t при трассировке.

Допуски логики (FeEps)

- `SamePointEps` $= 10^{-6}$ — совпадение точки со стеной/вершиной.
- `NoMoveEps` $= 10^{-9}$ — порог «луч никуда не попал».
- `WallScanSteps` $= 120$ — точек на зеркале при расчёте минимального расстояния.

Пределы (Limits)

- `MinX..MaxX` $= 0..1580$, `MinY..MaxY` $= 0..1000$ — допустимые координаты.
- `MinPointDist` $= 10$ — минимальное расстояние между вершинами.
- `MinWalls`/`MaxWalls` $= 4/9$ — допустимое число сторон.
- `ClickRadius` $= 5$ — попадание клика в точку/источник.
- `BeamWallTolerance` $= 6$ — допуск постановки источника.
- `MinBeamVertexDist` $= 10$ — минимум от источника до вершины.
- `MaxMirrorRadius` $= 1\,000\,000$ — предел радиуса кривой стены (и в диалоге, и при загрузке).
- `DefaultBeamAngle` $= 90°$ — угол луча по умолчанию, если в файле не задан.
- `MaxLoadFileBytes` $= 1$ МиБ — предельный размер файла для загрузки.

Где какая формула применяется

- **Отражение** $\mathbf{d}' = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\mathbf{n}$ — в `Polygon::TraceBeamIter`.
- **Нормаль к плоской стене** — в `GetNormal`. Перпендикуляр к хорде, нормирован.
- **Нормаль к дуге** — в `GetNormal`. Вектор от центра к точке попадания.
- **Центр дуги** $C = M \pm h\cdot\mathbf{n}_\perp$, $h = \sqrt{r^2 - (d/2)^2}$ — в `Mirror::CircleCenter`.
- **Пересечение луча с отрезком** — в `RaySegmentIntersect`.
- **Пересечение луча с дугой** — в `RayArcIntersect`: $\|P + t\mathbf{d} - C\|^2 = r^2$.
- **Ближайшая точка** — в `Mirror::ClosestPoint`, t^* зажимается в $[0, 1]$.
- **Проверка** $|r| \geq d/2$ — в конструкторе `Mirror` и в `MakeMirror`.

8.7 Алгоритм загрузки комнаты из файла

Загрузка выполняется в два шага: `MainWindow::LoadRoomFromFile` читает файл и передаёт содержимое в `FrontEndOperators::LoadRoom`, который разбирает и проверяет его, а затем строит готовый `Polygon`.

MainWindow::LoadRoomFromFile().

1. Открывает `QFileDialog` с фильтром «Текстовые файлы (*.txt)»; папка — последняя загрузка, иначе последнее сохранение, иначе «Документы».
2. Отмена диалога — сообщение «Загрузка из файла отменена.», ничего не меняется.
3. Проверяет размер: не больше `Limits::MaxLoadFileBytes`.
4. Читает файл целиком через `QFile::readAll`.
5. Передаёт `std::string text` в `LoadRoom`.

6. При успехе вызывает `SecondButClicked()` — тот же путь, что и после ручного построения.

FrontEndOperators::LoadRoom().

1. **Этап 0.** UTF-8 → UTF-32 (`DecodeUtf8`, повреждённые байты → U+FFFD), отбрасывание BOM, разбиение на строки (`SplitLines`). Пустой файл — ошибка «Файл пуст». Поиск первых строк с фразами «Углы комнаты:» (`HasPhrase({U"углы U"комнаты"})`) и «Луч:» (`HasPhrase({U"луч"})`).

2. **Этап 1 (структура).** Проверка фразы углов; `ParseRoomLine` читает до 9 значащих строк; подсчёт пропущенных сверх 9. Для луча — ровно 2 или 3 числа (`ExtractNumbers`) и угол в `[BeamAngle::Min, BeamAngle::Max]`. Все ошибки собираются в вектор и выводятся сразу.

3. **Этап 2 (значения).** Каждая точка в рабочей области; все пары вершин на `MinPointDist`; для кривых стен — $\frac{1}{2}$ хорды $\leq R \leq$ `MaxMirrorRadius`.

4. **Этап 3 (геометрия).** Строится временный `Polygon tmp`; `MakeMirror` может бросить `std::invalid_argument`. Проверяются `CheckWalls(tmp, closed=true)` (пересечения и близость) и `Polygon::Check()`.

5. **Луч.** `PlaceBeam` (допуск 6 пикселей, минимум 10 до вершины), затем `SetBeamAngle`. Неудача здесь не отклоняет комнату: результат `RoomOnly`.

6. **Замена.** `prototype = std::move(tmp)` — комната подменяется целиком, старый источник исчезает.

Возвращаемый результат. `LoadResult::Failed` (ошибка, ничего не изменилось), `LoadResult::RoomOnly` (комната без луча), `LoadResult::RoomWithBeam` (комната и луч).

9 Заключение

Проект **MirrorRoomSimulation** — законченное интерактивное приложение для моделирования отражений луча в зеркальной комнате. Он содержит пользовательский интерфейс, геометрическую модель, алгоритмы трассировки, анимацию, служебные сообщения, экспорт истории и загрузку комнаты из текстового файла.

Итог

Для пользователя программа предоставляет понятный сценарий: построить комнату (мышью или из файла), задать зеркала, поставить источник, выполнить отражения и сохранить результат. Для программиста проект показывает пример организации Qt-приложения с разделением интерфейса, логики и геометрической модели и с атомарной загрузкой из файла.